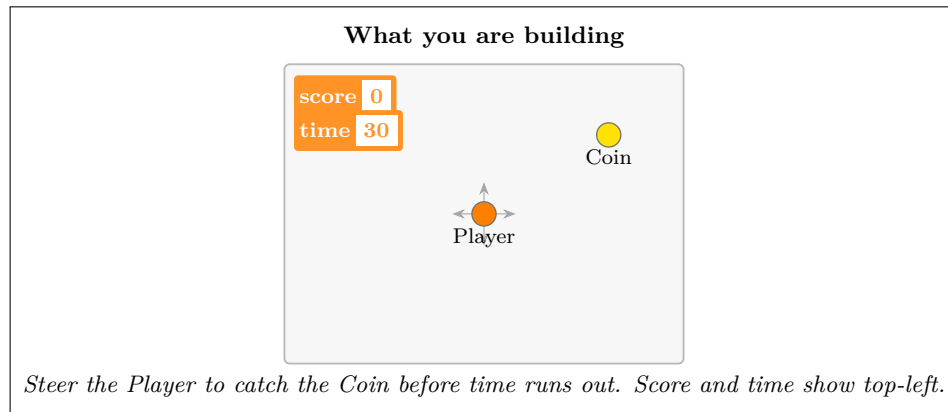


Answer key — finished scripts and challenge solutions.

Welcome to Scratch E - Coin Dash. You have all the blocks now. This sheet builds *one* game in three stages: a player you steer to catch coins before the clock runs out. Most of each script is given, ready to drag together. But watch for the grey ? **your turn** blocks: each one is a gap *you* must fill with the right block. The finished picture shows what to build. The grey block tells you what is missing, in plain words. Every missing block is one you have used before. Open scratch.mit.edu, click *Create*, and let us play.



The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: Steer the player, drop a coin

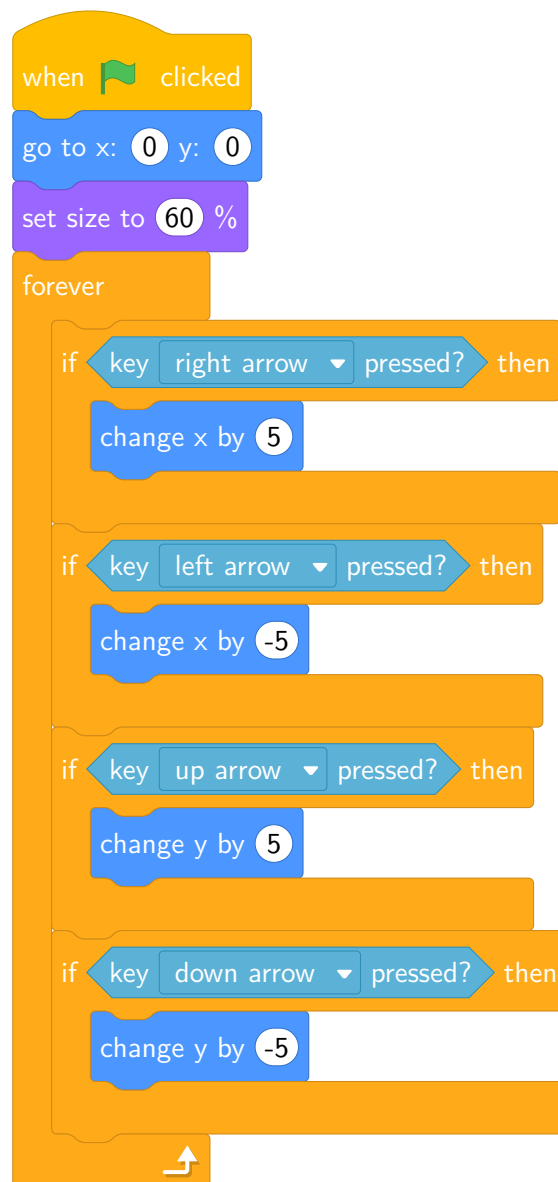
Steer the cat with the arrow keys, the smooth way: one forever loop that reads all four keys. Then add a Coin sprite that starts at a random spot on the stage.

Get ready. Rename the cat sprite to **Player**. Then add a coin: click *Choose a Sprite*, pick a ball or a doughnut, and rename it **Coin**.

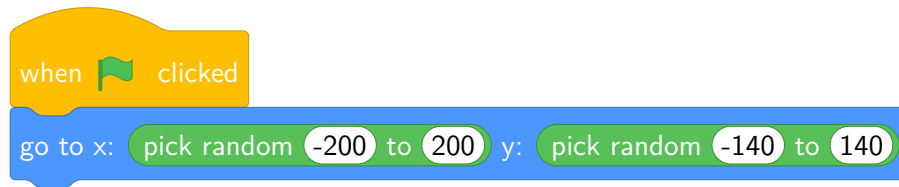
Build it.

1. On the **Player**, start with `when green flag clicked`, then `go to x: 0 y: 0` and `set size to (60) %`.
2. Add a `forever` loop. Inside it, add four choices, one per arrow key: `if <key [right arrow] pressed?>` then `change x by (5)`; `left arrow` with `change x by (-5)`; `up arrow` with `change y by (5)`; `down arrow` with `change y by (-5)`.
3. Click the green flag and steer with the arrow keys.
4. Now click the **Coin**. Give it `when green flag clicked`. The last block is your first `? your turn`: put the coin at a random spot so it is not in the same place every game.

The Player's steering script, then the Coin's start. The grey block is yours to fill:



Player: steer with the arrows



Coin: start somewhere random

The stage runs from x: -240 to 240 and y: -180 to 180. Staying inside about -200 to 200 and -140 to 140 keeps the coin fully on screen.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Change `set size to (60) %` so the player is the size you like. Try 40 for a tiny, nippy cat.

Expected: A smaller player is easier to squeeze through the stage.

2. Slow the player down or speed it up: change every `5` and `-5` in the loop to a number you like.

Expected: Bigger steps move faster; smaller steps are easier to control.

Project 2: Grab it, score up

Make a `score`. When the player touches the coin, the score goes up by one and the coin jumps to a fresh random spot, ready to be caught again.

Get ready. Make a variable `score` For all sprites (Variables blocks, *Make a Variable*). It will show in the top-left of the stage.

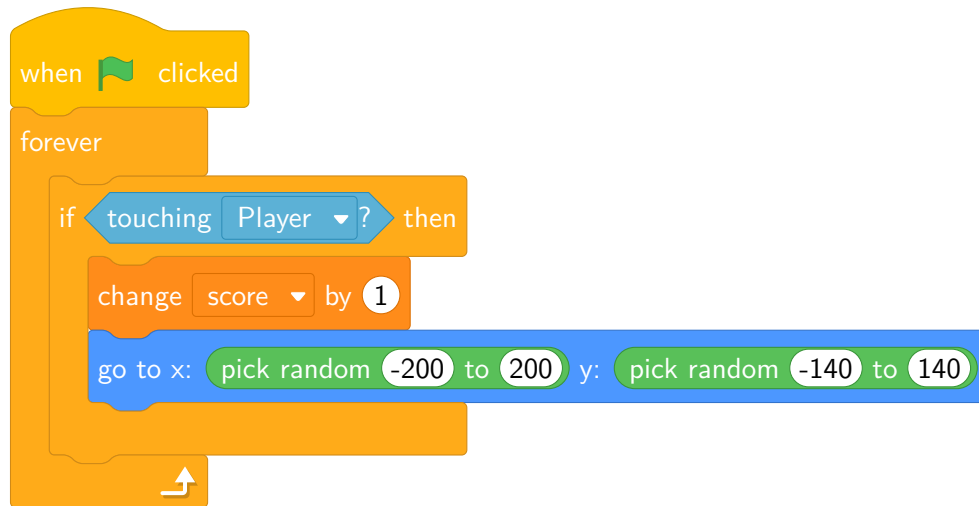
New blocks.

`if <touching [Coin]?>` True when the player is touching the coin. Here it is on the Coin, so it checks touching the Player.

Build it.

1. On the **Player**, in the `when green flag clicked` script, add `set [score] to (0)` near the top, before the forever loop.
2. Now click the **Coin** and add a new script: `when green flag clicked` then a `forever` loop.
3. Inside the loop, add `if <touching [Player]?>`. That is when the coin has been caught.
4. Inside that `if`, the two blocks are `? your turn`: first add a point to the score, then send the coin to a new random spot.
5. Green flag, and chase the coin. Each catch adds one and the coin hops away.

The Coin's catch loop. Both blocks inside the `if` are yours to fill:



Coin: catch and re-spawn

Setting `score` to 0 on the Player's green-flag script means every new game starts from zero. The coin's job is only to add points.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Give catching a sound: on the Coin, click the *Sounds* tab, add a *Coin* or *Pop* sound, then put `start sound [Pop]` as the first block inside the `if`.

Expected: A sound plays the instant the coin is caught.

2. Make some coins worth more: change the point block to `change [score] by (pick random (1) to (3))`.

Expected: Each catch is now worth 1, 2, or 3 points at random.

Project 3: Mini project: beat the clock

Add a countdown. A `time` variable starts at 30 and drops by one each second. When it reaches zero the game stops, and your final `score` is your result.

Get ready. Make a second variable `time` For all sprites. You will build the countdown as a new script on the **Player**.

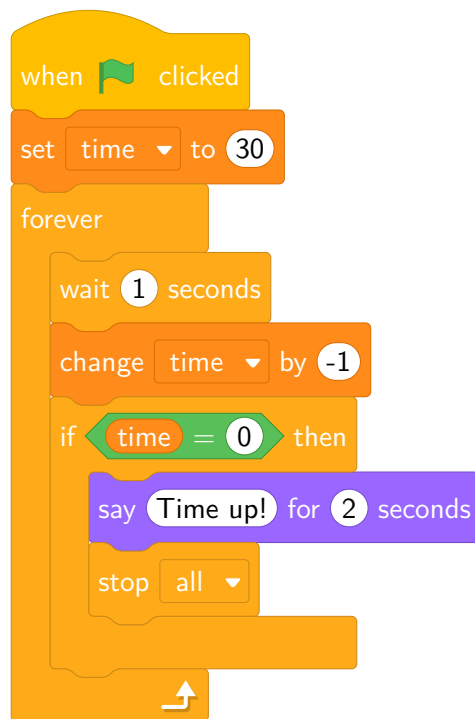
New blocks.

`if <(time) = (0)>` True the moment the countdown reaches zero. It compares the time variable with 0.

Build it.

1. On the **Player**, add a new script: `when green flag clicked` then `set [time] to (30)`.
2. Add a `forever` loop. Its first block is `wait (1) seconds`, so the loop ticks once a second.
3. The next block is `? your turn`: count the time down by one on every tick.
4. The last block is an `if` that ends the game. Its shell and its inside blocks (`say (Time up!)` and `stop [all]`) are given. The `? your turn` is the grey hexagon: fill in the test that checks the time has reached zero.
5. Green flag. Grab as many coins as you can before the clock runs out.

The countdown loop. Fill the grey stack block and the grey hexagon in the `if`:



Player: the 30-second countdown

The countdown lives in its own forever loop with `wait (1) seconds`, so it ticks steadily while your other loops handle moving and catching at full speed.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Show a start message: on the Player, add `say (Catch the coins!) for (2) seconds` right after `set [time] to (30)`.

Expected: The player explains the goal at the start of each game.

2. Keep a high score. Make a variable `best`. Just before `stop [all]`, add `if <(score) > (best)>` then `set [best] to (score)`.

Expected: The best score so far stays on screen across games.

3. Add more time for a longer game: change `set [time] to (30)` to a bigger number, like 60.

Expected: A longer countdown means a higher possible score.